

Il test reale su dual boot

Le slide una alla volta, ognuna con la sua spiegazione

Ogni pagina ha l'immagine di una slide e sotto la spiegazione. I riquadri grigi sono **esempi con i numeri**; i riquadri gialli segnalano i **punti da correggere o da dire con cautela**; i riquadri verdi le **verifiche** fatte ricontrollando i dati.

Le parole e le unità che tornano sempre

Nodo	Il computer che riceve il pacchetto, esegue la rete neurale e lo inoltra. Nel banco è il core 6.
XDP	Il primo punto del kernel Linux in cui un programma può vedere un pacchetto appena arrivato.
Pipeline	Una delle versioni del programma che esegue la rete neurale (P1, P1.5, P2, P3).
baseline / rxonly	I due programmi di riferimento: la baseline inoltra senza rete neurale, rxonly riceve e basta.
Coda	I 256 posti davanti al nodo dove i pacchetti aspettano. Se è piena, chi arriva è respinto.
Coda d'uscita	La coda del cavo virtuale verso il nodo successivo. Con più core sul nodo ce n'è una per core, svuotata da un core suo (slide 14).
Capacità	Quanti pacchetti al secondo il nodo riesce a elaborare al massimo.
Collo di bottiglia	Il pezzo più lento della catena: decide quanto traffico passa.
M/s	Milioni di pacchetti al secondo.
ns, μ s, ms	Nanosecondi (miliardesimi di secondo), microsecondi ($1\ \mu\text{s} = 1\ 000\ \text{ns}$), millisecondi ($1\ \text{ms} = 1\ 000\ \mu\text{s}$).
Gbit/s	Miliardi di bit al secondo. Con pacchetti da 64 byte, $1\ \text{Gbit/s} \approx 1,95$ milioni di pacchetti al secondo.
Costo per pacchetto	1 diviso la capacità. 2 milioni di pacchetti al secondo = 500 ns per pacchetto.
Tempo di CPU per pacchetto	Quanto tempo il core del nodo lavora davvero per ogni pacchetto: occupazione del core $\times$ 1 diviso la capacità. Coincide con il costo per pacchetto quando il nodo lavora il 100% del tempo.
Scrittore	Chi mette pacchetti in una coda. Con una scheda di rete vera è la scheda stessa: uno per coda.

TESI · EBPF / XDP

Il test reale su dual boot e la domanda sul bit rate

Come si misura quanti pacchetti regge un nodo, con quali generatori di traffico, e che cosa succede quando il traffico offerto cresce.

Ettore Cantile
settembre 2026

La presentazione racconta il test del datapath su traffico vero, rifatto sul portatile con Ubuntu installato direttamente (dual boot) invece che dentro una macchina virtuale. Al centro c'è il **nodo**: il computer che riceve un pacchetto, esegue la rete neurale dentro il kernel (con XDP, cioè nel primo punto in cui Linux vede il pacchetto) e decide da quale porta farlo uscire.

Le domande a cui risponde sono tre:

1. Quanti pacchetti al secondo regge un nodo, con ciascuna delle versioni del programma?
2. Con quale traffico lo misuriamo? Ci sono due generatori diversi, pktgen e xdp_gen, e danno numeri diversi per ragioni precise.
3. Che cosa succede quando il traffico cresce oltre quello che il nodo regge: dove si perdono i pacchetti? È la domanda del relatore, e la risposta arriva nella parte 5.

Tutte le cifre vengono dallo stesso portatile (Intel Core Ultra 7 155H), con le condizioni della macchina controllate e registrate a ogni misura. Le misure a un core sono del 28-30 settembre, quelle a due core del 1 ottobre.

La domanda del relatore



Se si manda al nodo sempre più traffico, a un certo punto il nodo non ce la fa più. Davanti al nodo c'è una **coda** di 256 posti: i pacchetti che il nodo non ha ancora elaborato aspettano lì, in ordine di arrivo. Finché il nodo è più veloce del traffico la coda resta quasi vuota; quando è più lento la coda si riempie, e i pacchetti che arrivano a coda piena vengono **respinti**, cioè persi.

La slide segue questa catena in quattro passi: il traffico cresce, la coda si riempie, il ritardo cresce (ogni pacchetto aspetta quelli davanti), e alla fine si perde. Il relatore chiede di capire **dove** si perde, perché il punto della perdita dice chi è il collo di bottiglia:

- se si perde **prima** del programma (il pacchetto non è mai entrato), il programma è troppo lento per il traffico;
- se si perde **dopo** (il programma lo ha elaborato e deciso di inoltrarlo, ma non arriva a destinazione), il problema è la trasmissione in uscita.

Esempio. Il nodo riesce a elaborare 1,9 milioni di pacchetti al secondo e il generatore ne manda 2,8 milioni. Ogni secondo 0,9 milioni di pacchetti non trovano posto e vengono respinti. Se contiamo quanti pacchetti entrano nel programma (1,9 milioni) e quanti ne escono (sempre 1,9 milioni), vediamo che si perdono tutti prima: il collo di bottiglia è il programma.

Per rispondere servono quindi due conteggi in più rispetto a quelli del generatore: i pacchetti che il programma riceve e quelli che rilancia. Come si ottengono è nelle slide 16 e 17.

Come è organizzata la presentazione

COME È FATTA LA PRESENTAZIONE

Cinque parti, dal banco alla risposta

Ogni parte risponde a una domanda: le prime quattro preparano la risposta alla quinta.

1

Il banco

Che cosa si misura, con quali programmi e come.

Che cosa **misuriamo**?

2

La macchina

Dalla macchina virtuale al dual boot.

Perché fidarsi dei **numeri**?

3

I generatori

pktgen e xdp_gen: due modi di produrre traffico.

Con che **traffico** si misura?

4

I risultati

Quanto regge il nodo, con l'uno e con l'altro.

Quanti **pacchetti** al secondo?

5

La domanda

Che cosa succede quando il traffico cresce.

Dove si **perde**?

IN BREVE

La risposta, in anticipo: si perde **prima** del programma, quindi il collo di bottiglia è **il programma**. E le misure ora si ripetono.

Le parti 2-5 raccontano ciò che è cambiato dal test in macchina virtuale

3

È la mappa delle cinque parti. Ognuna risponde a una domanda, e le prime quattro servono a potersi fidare dei numeri della quinta.

1. **Il banco** (slide 4-5): quali programmi si confrontano e come si misura il costo per pacchetto.
2. **La macchina** (slide 6-8): perché nella macchina virtuale le misure non erano affidabili, che cosa è cambiato con il dual boot e quanto sono ripetibili ora.
3. **I generatori** (slide 9-11): i due modi di produrre traffico, che cosa costa ciascuno al nodo e a che cosa serve.
4. **I risultati** (slide 12-15): quanti pacchetti al secondo regge ogni programma con l'uno e con l'altro generatore, e che cosa succede con due core.
5. **La domanda** (slide 16-23): dove si perde quando il traffico cresce, che cosa succede al ritardo, e come cresce la capacità con i core.

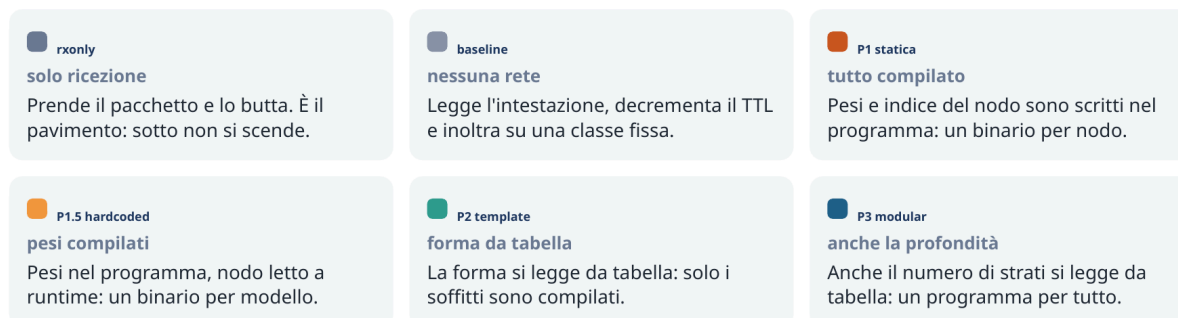
La risposta è anticipata nel riquadro in basso: si perde **prima** del programma, quindi il collo di bottiglia è il programma eBPF, non la trasmissione. E, grazie al dual boot, le misure ora si ripetono.

I sei programmi

1 - IL BANCO - I PROGRAMMI

Sei programmi, che cambiano per una cosa sola

Calcolano la stessa rete e prendono la stessa decisione. Cambia quanto sanno già **quando vengono compilati**.



Più informazioni si spostano dalla compilazione all'esecuzione, più il programma è **riutilizzabile**, e più cresce il **costo per pacchetto**.

Cinque pipeline più rxonly, che misura la sola ricezione - i colori dei quadratini tornano in tutti i grafici

4

Le quattro pipeline (P1, P1.5, P2, P3) calcolano la stessa rete neurale e prendono la stessa decisione sullo stesso pacchetto: lo verificano i test di correttezza. Cambia solo **quanto è scritto dentro il programma** quando lo si compila, e quanto invece il programma legge da tabelle (le mappe eBPF) mentre gira. Due programmi di riferimento completano il confronto.

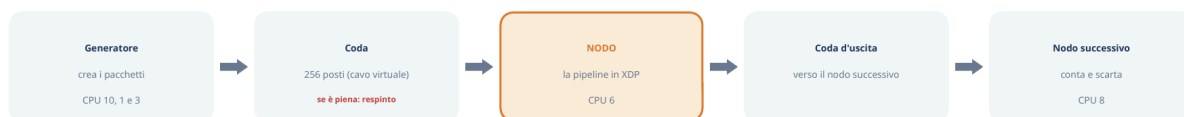
- **rxonly**: riceve il pacchetto, incrementa un contatore e lo butta. Non fa altro. Serve a sapere quanto costa la sola ricezione: nessun programma può essere più veloce, quindi è il "pavimento".
- **baseline**: legge l'intestazione, abbassa il TTL e inoltra il pacchetto su una porta fissa, senza rete neurale. Fa tutto quello che fa una pipeline tranne l'inferenza: per questo **quanto una pipeline costa in più della baseline è il costo della rete neurale**.
- **P1 statica**: pesi della rete e numero del nodo sono costanti nel codice. Il compilatore può semplificare molto (moltiplicazioni per costanti, rami eliminati), ma serve un binario per ogni nodo: quello compilato per il nodo 7 non va bene per il nodo 8.
- **P1.5 hardcoded**: i pesi sono nel codice, il numero del nodo si legge da una tabella. Un binario per modello, valido per tutti i nodi che usano quel modello.
- **P2 template**: anche la forma della rete (quanti ingressi, quanti neuroni per strato) si legge da tabella. Nel codice ci sono solo i soffitti, cioè i valori massimi ammessi, e i cicli girano fino a quelli. Un solo programma serve molti modelli.
- **P3 modular**: anche il numero di strati si legge da tabella, e ogni strato è un programma a parte raggiunto con un salto (tail call). È la versione più generale.

Regola generale: più informazioni si spostano dalla compilazione all'esecuzione, più il programma è riutilizzabile, e più lavoro fa per ogni pacchetto (letture da tabella, cicli più lunghi, salti). Le slide successive misurano proprio quanto costa questa generalità. I colori dei quadratini restano gli stessi in tutti i grafici.

Il banco di misura

1 - IL BANCO - COME SI MISURA

Il banco misura quanti pacchetti regge il nodo



1

Coda sempre piena

Il generatore offre **più pacchetti** di quanti il nodo riesca a elaborare.

2

Nodo sempre occupato

Il suo core non resta mai senza lavoro: il ritmo a cui elabora è la sua **capacità**.

3

Costo = 1 / capacità

Se elabora 5 milioni di pacchetti al secondo, spende **200 ns** per ciascuno.

QUINDI

Si misurano 300 ms a regime: costo per pacchetto = tempo ÷ pacchetti elaborati. Il generatore sta su altri core, quindi **non entra nella misura**.

Un solo portatile: generatore, nodo e nodo successivo ciascuno sul proprio core fisico

5

Tutto gira sullo stesso portatile, ma ogni pezzo ha il suo core fisico dedicato, così nessuno ruba tempo agli altri:

- il **generatore** (CPU 10, 1 e 3) crea i pacchetti;
- la **coda** da 256 posti è quella del cavo virtuale (veth) che collega generatore e nodo;
- il **nodo** (CPU 6) esegue la pipeline in XDP: è l'unica cosa che vogliamo misurare;
- la **coda d'uscita** porta i pacchetti inoltrati verso il nodo successivo;
- il **nodo successivo** (CPU 8) conta i pacchetti che arrivano e li butta.

Come si misura il costo. Si manda più traffico di quanto il nodo possa elaborare. Così valgono le tre condizioni della slide: la coda è sempre piena, il core del nodo non resta mai senza lavoro, e il ritmo a cui elabora è la sua **capacità**. Il tempo speso per ogni pacchetto è allora 1 diviso la capacità.

Esempio. In una finestra di 300 millisecondi il nodo elabora 900 000 pacchetti. Sono 3 milioni al secondo, quindi spende 1 / 3 000 000 di secondo per pacchetto, cioè circa 333 ns.

Perché mandare più traffico del necessario? Se il generatore mandasse meno pacchetti di quanti il nodo ne regge, il nodo starebbe fermo ogni tanto e misureremmo il ritmo del generatore, non quello del nodo. La misura avviene solo a regime: si scartano i primi istanti, si leggono i contatori a inizio e fine finestra, e si ripete più volte (giri e ripetizioni) per vedere quanto varia.

Aggiunto il 30 settembre. Il banco misura anche quanto è occupato il core del nodo, nella stessa finestra in cui conta i pacchetti. Con pktgen è occupato al 100%, e 1 diviso la capacità è davvero il suo costo. Con xdp_gen a un thread il nodo a tratti aspetta (slide 13): lì il costo si calcola come **tempo di CPU per pacchetto**, cioè occupazione del core per 1 diviso la capacità.

Il problema della macchina virtuale

2 · MACCHINA VIRTUALE E DUAL BOOT · IL PROBLEMA

Nella macchina virtuale il nodo ogni tanto si fermava

Quanto può stare fermo il nodo

prima che la coda da 256 posti trabocchi

Riempire la coda (1,5 M pacchetti/s)

Risveglio da sonno profondo

Pausa di un core virtuale

la coda trabocca

170 μ s310 μ s

alcuni millisecondi

Due strade, entrambe cattive

Generatore e nodo su core separati

Il nodo perdeva **fra l'8 e il 70%** dei pacchetti a qualunque ritmo, anche lentissimo.

Generatore e nodo sullo stesso core

Non si perdeva niente, ma il core faceva il lavoro di tutti e due: il collo di bottiglia era sempre **il generatore**.

QUINDI

La perdita misurava **le assenze del core**, non il programma: dentro la macchina virtuale un core può fermarsi per millisecondi, e non si vede né si può impedire.

6

Fino al 26 settembre il test girava dentro VirtualBox. Un "core" della macchina virtuale non è un core vero: è un thread del sistema che ospita la macchina virtuale, e quel sistema può fermarlo quando vuole, anche per qualche millisecondo. Dall'interno non lo si vede e non lo si può impedire.

Il grafico a sinistra mette a confronto i tempi in gioco. Quanto può stare fermo il nodo prima di perdere pacchetti? Dipende da quanto ci mette la coda a riempirsi. A 1,5 milioni di pacchetti al secondo ne arriva uno ogni 0,67 μ s, quindi 256 posti si riempiono in $256 \times 0,67 \approx 170 \mu$ s. Qualunque pausa più lunga fa traboccare la coda. Un core che si risveglia dal sonno profondo ci mette 310 μ s; una pausa del core virtuale può durare millisecondi: entrambe superano di molto il margine.

Esempio. Il core virtuale viene fermato per 2 millisecondi. In quel tempo arrivano $2 \text{ ms} \times 1,5 \text{ milioni/s} = 3 \text{ 000}$ pacchetti, ma la coda ne contiene 256: circa 2 750 vanno persi. E il programma non c'entra niente: poteva essere velocissimo.

Si poteva scegliere solo fra due strade, entrambe sbagliate:

- **generatore e nodo su core separati:** perdite dall'8 al 70% anche a traffico bassissimo, cioè si misuravano le pause del core e non il programma;
- **tutto sullo stesso core:** niente perdite, perché il core non si ferma mentre lavora per sé, ma il core faceva il lavoro del generatore e del nodo insieme, e il limite era sempre il generatore.

La conclusione è che nella macchina virtuale la perdita misurava le assenze del core. Per rispondere alla domanda del relatore serviva una macchina vera.

La soluzione: il dual boot

2 · MACCHINA VIRTUALE E DUAL BOOT · LA SOLUZIONE

Sul dual boot ogni core ha un ruolo fisso

La macchina: un processore ibrido

Intel Core Ultra 7 155H, 22 CPU: ognuna ha il suo ruolo



E-core 12-19 e LP E-core 20-21: sistema

In tratteggio: il thread gemello di ogni core, lasciato a riposo.

1 Frequenza fissa e misurata

3,5 GHz richiesti, 3 494 MHz misurati dai contatori del processore durante ogni misura.

2 Niente sonno profondo

Un core che dorme impiega 310 μ s a svegliarsi, più dei 170 μ s che la coda concede.

3 Il resto del sistema altrove

Desktop, servizi, interruzioni e thread del kernel stanno sulle altre CPU.

4 Un monitor sempre acceso

Temperatura, throttling e frequenza vera: una finestra fuori condizione si conta, non si scarta.

QUINDI

Il core del nodo **non dorme, non cambia frequenza e nessuno lo interrompe**: si misura il programma, non la macchina.

7

Dal 27 settembre Ubuntu gira direttamente sul portatile. Lo script `host_conditions.py` prepara la macchina prima di ogni misura e la rimette com'era alla fine (anche se la misura si interrompe):

- **ogni ruolo ha un core fisico suo** (schema a sinistra). Il processore è ibrido: 6 P-core veloci, ciascuno con due thread gemelli (SMT), più 8 E-core e 2 LP E-core più lenti. Il gemello di ogni core usato resta a riposo, perché due thread sullo stesso core fisico si dividono pipeline e cache e si rallentano a vicenda. Desktop e servizi vanno sugli E-core;
- **frequenza fissa**: 3,5 GHz richiesti, e si verifica con i contatori del processore (APERF/MPERF) che sia davvero quella: 3 494 MHz misurati durante le misure. Senza frequenza fissa il turbo cambierebbe la velocità del core da un istante all'altro;
- **niente sonno profondo**: gli stati di risparmio energetico profondi (C6, C10) sono spenti sui core del banco, perché un core che si addormenta ci mette fino a 310 μ s a svegliarsi, più dei 170 μ s che la coda concede (slide 6);
- **tutto il resto altrove**: processi, thread del kernel, interruzioni e code di lavoro del sistema vengono spostati sulle CPU di sistema;
- **un monitor sempre acceso**: ogni secondo registra temperatura, throttling termico e frequenza vera. Se una finestra di misura avviene in condizioni sbagliate non si scarta di nascosto: viene contata e segnalata nel file `env.csv` del run.

Risultato: il core del nodo non dorme, non cambia velocità e nessuno lo interrompe. Si misura il programma, non la macchina.

Prima e dopo

2 · MACCHINA VIRTUALE E DUAL BOOT · PRIMA E DOPO

Con il dual boot le misure si ripetono

Macchina virtuale		Dual boot		
Pipeline	costo	variazione	costo	variazione
P1 statica	~82 ns	160%	38 ns	0,5%
P1.5	~116 ns	104%	53 ns	0,6%
P2	~430 ns	19%	226 ns	0,5%
P3	~670 ns	15%	364 ns	0,3%

Costo sopra la baseline, per pacchetto. Le cifre assolute non si confrontano (frequenza e metodo erano diversi): si confronta la variazione.

160% → 0,5%

variazione del costo di P1 fra le esecuzioni

1-2% → 0

pacchetti respinti sotto capacità, anche per il solo contatore

58% · 0,2% · 7,7%

perdita dello stesso punto in tre giri (macchina virtuale). Sul dual boot: entro lo 0,4-2%

QUINDI

Il guadagno vero non è la velocità, è la **ripetibilità**: differenze che prima sparivano nel rumore, come P1 contro P1.5 o scartare contro inoltrare, ora si misurano.

Macchina virtuale: 18 settembre, 512 byte · dual boot: 28 settembre, core separati, 64 byte · variazione fra le esecuzioni

8

Da quale test vengono i numeri. La tabella viene dal test di throughput con traffico vero e generatore pktgen (lo stesso della slide 12). "Costo" è quanti nanosecondi per pacchetto una pipeline spende in più della baseline, cioè il costo della rete neurale. "Variazione" dice quanto cambia il risultato ripetendo la stessa misura. Le cifre assolute delle due colonne non si confrontano, perché frequenza, taglia del pacchetto e metodo erano diversi: si confronta la variazione.

Esempio. Nella macchina virtuale P1 una volta costava 164 ns e un'altra 32 ns: variazione del 160%. Con numeri così non si può dire se P1 sia più veloce di P1.5, che costa circa 15 ns in più. Sul dual boot le ripetizioni danno quasi sempre lo stesso valore, e la differenza fra P1 e P1.5 si vede.

Gli altri numeri a destra:

- **160% → 0,5%:** la variazione del costo di P1 fra le esecuzioni, prima e dopo;
- **1-2% → 0:** con un traffico più basso della capacità, nella macchina virtuale si perdeva comunque l'1-2% dei pacchetti (per le pause del core), anche con il solo contatore; sul dual boot nessuno;
- **58% · 0,2% · 7,7%:** nella macchina virtuale lo stesso punto del test sul bit rate, ripetuto tre volte, dava tre perdite completamente diverse. Sul dual boot lo stesso punto varia dello 0,4-2%.

Il messaggio della slide: il guadagno vero del dual boot non è la velocità, è la **ripetibilità**. Differenze che prima sparivano nel rumore, come P1 contro P1.5 o scartare contro inoltrare (slide 13), ora si misurano.

Attenzione. Le due colonne "variazione" non misurano esattamente la stessa cosa: quella della macchina virtuale riguarda il costo sopra la baseline, quella del dual boot riguarda i pacchetti al secondo di ogni pipeline. Misurata come per la macchina virtuale, la variazione sul dual boot fra sessioni diverse è circa 1,6 / 3,2 / 2,5 / 3,4% (P1 / P1.5 / P2 / P3). Il miglioramento resta enorme (P1: da 160% a circa 2%), ma lo 0,5% della slide lo fa sembrare più grande di quanto sia.

Il primo generatore: pktgen

3 · I DUE GENERATORI DI TRAFFICO · PKTGEN

pktgen produce i pacchetti come li produce Linux



≈ **5,7 M/s** pacchetti al secondo offerti da tre thread su tre core

≈ **4,6 M/s** quanti ne accetta la coda del nodo: lo misura rxonly

QUINDI

La copia è una proprietà del kernel, **non della macchina virtuale**: il dual boot non la toglie. Su una scheda di rete con XDP nativo non esisterebbe.

La baseline costa 302 ns per pacchetto con pktgen · M/s = milioni di pacchetti al secondo

9

pktgen è un generatore di traffico che sta dentro il kernel Linux. Crea i pacchetti come li crea il resto del sistema: dentro una struttura chiamata **skb** (socket buffer), che descrive il pacchetto e lascia 64 byte liberi davanti ai dati.

XDP però vuole 256 byte liberi davanti al pacchetto: servono a un programma che volesse aggiungere intestazioni senza spostare i dati. Il cavo virtuale, prima di consegnare il pacchetto al nodo, deve quindi **copiarlo** in una pagina nuova con lo spazio giusto. La slide segue i tre passi: pktgen crea la skb, il cavo la copia, il nodo la elabora. Questa copia la fa il core del nodo, prima ancora che il programma parta, quindi finisce nel tempo che misuriamo.

Esempio. Con pktgen P2 costa 529 ns per pacchetto; con l'altro generatore, che non richiede la copia, 293 ns di CPU. Circa 230 ns del tempo misurato con pktgen sono quindi trasporto e copia, non lavoro della pipeline, e sono gli stessi per tutte le pipeline (slide 15).

Il passaggio al dual boot non toglie la copia: è un comportamento del kernel con i cavi virtuali, non della macchina virtuale. Su una scheda di rete vera con XDP nativo non ci sarebbe, perché la scheda scrive il pacchetto direttamente in una pagina con lo spazio giusto.

I due numeri in basso: con tre thread su tre core pktgen offre circa **5,7 milioni** di pacchetti al secondo, e la coda del nodo ne accetta circa **4,6**, che è quanto regge la sola ricezione (rxonly). La baseline, con la copia, costa 302 ns per pacchetto.

Il secondo generatore: xdp_gen

3 · I DUE GENERATORI DI TRAFFICO · XDP_GEN

xdp_gen consegna pacchetti già nel formato di XDP



12-15

milioni di pacchetti al secondo
offerti da un solo thread

293 ns

P2 con xdp_gen, contro 529 ns
con pktgen

QUINDI

Il costo del generatore non passa più dal nodo. Si usa **un thread per coda**: più thread nella stessa coda si intralciano e rallentano anche il nodo (slide 13).

Frame grezzi creati con BPF_PROG_TEST_RUN in modalità live frames · script ipa/test/xdp_gen.py

10

xdp_gen (script `ipa/test/xdp_gen.py`) usa una funzione del kernel, `BPF_PROG_TEST_RUN` in modalità **live frames**: il kernel prepara dei pacchetti già nel formato che XDP si aspetta (un frame grezzo con i 256 byte liberi davanti) e li spedisce davvero sul cavo virtuale. Il frame arriva al nodo così com'è: niente skb e niente copia, come arriverebbe da una scheda di rete vera con XDP nativo.

Esempio. Lo stesso programma, P2, costa 529 ns con pktgen e 293 ns di CPU con xdp_gen. Il programma non è cambiato: è cambiato solo il modo in cui il pacchetto arriva al nodo.

Un solo thread di xdp_gen offre 12-15 milioni di pacchetti al secondo, tre volte quanto offrono tre thread di pktgen. Si usa così: **un thread per ogni coda del nodo**. La coda in cui un thread scrive dipende dalla CPU su cui gira (numero della CPU modulo numero di code), e con più thread nella stessa coda i thread si intralciano e rallentano anche il nodo (slide 13).

Verificato il 30 settembre. Con tre thread il nodo si fermava intorno a 7 milioni al secondo, e sembrava il tetto del generatore. Non lo era: con **un thread** la baseline elabora 12,4 milioni e rxonly 14,9. Il limite era la coda d'ingresso condivisa da tre scrittori (slide 13).

I due generatori a confronto

3 · I DUE GENERATORI DI TRAFFICO · A CONFRONTO

Due generatori, per due scopi diversi

Stesso nodo, stessi programmi: cambia solo **come arriva il pacchetto**.

Confronto	pktgen	xdp_gen
Come crea il pacchetto	una skb, come lo stack di Linux	un frame grezzo, già nel formato di XDP
Copia prima del nodo	sì, a ogni pacchetto	no
Pacchetti offerti	≈ 5,7 M/s (il nodo ne accetta ≈ 4,6)	≈ 12–15 M/s (un thread)
Chi arriva al tetto	nessuna pipeline: tutte e cinque saturano sotto	baseline e rxonly; P1, P1.5, P2 e P3 saturano
Serve per	confrontare tutte le pipeline fra loro	misurare il costo di P1–P3 senza la copia

QUINDI Si usano **insieme**: ognuno confronta le pipeline al suo interno, e le cifre dei due generatori non si mescolano.

Il tetto di pktgen è il tetto di ricezione del nodo (rxonly, 4,6 M/s) · M/s = milioni di pacchetti al secondo

11

La tabella riassume le due slide precedenti, riga per riga:

- **come crea il pacchetto:** pktgen una skb, come lo stack di Linux; xdp_gen un frame grezzo, già nel formato di XDP;
- **copia prima del nodo:** con pktgen sì, a ogni pacchetto; con xdp_gen no;
- **pacchetti offerti:** circa 5,7 milioni al secondo con tre thread pktgen (il nodo ne accetta 4,6), 12–15 milioni con un solo thread xdp_gen;
- **chi arriva al tetto:** con pktgen nessuna pipeline, perché tutte e cinque saturano sotto quello che il generatore offre; con xdp_gen a un thread baseline e rxonly non saturano, P1–P3 sì;
- **serve per:** è la riga importante, sotto.
- **pktgen** è più lento, quindi tutte le pipeline arrivano al loro limite prima di lui. Si possono confrontare tutte e cinque fra loro, e anche rxonly satura. È la misura principale per il confronto fra pipeline.
- **xdp_gen** toglie il costo della copia, quindi dà il costo più vicino a quello che il nodo pagherebbe con una scheda vera. Con un thread per coda P1, P1.5, P2 e P3 saturano; baseline e rxonly no, e per loro il valore è indicativo. Con xdp_gen a un core si citano quindi P1–P3.

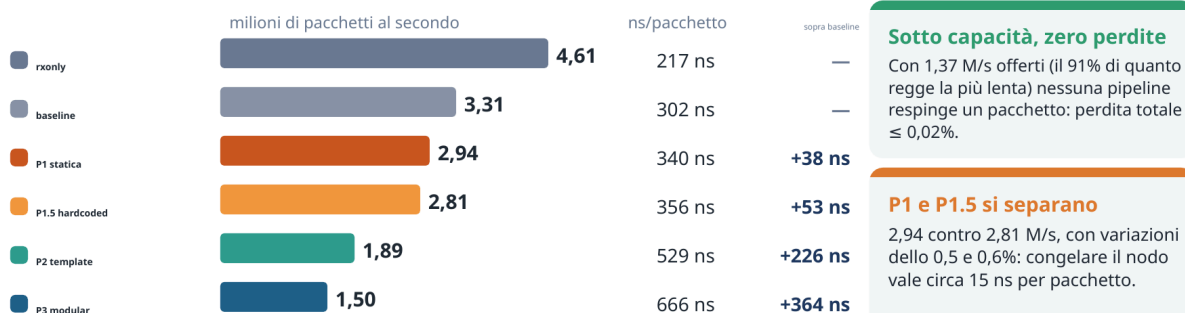
Esempio d'uso. Per dire "P1 è più veloce di P1.5 di circa 15 ns" si usa pktgen. Per dire "P2 costa circa 293 ns di CPU per pacchetto senza il sovraccarico della copia" si usa xdp_gen.

Le cifre assolute dei due generatori non si mescolano: ognuno si usa per confrontare le pipeline al suo interno. **Sopra la baseline**, però, danno lo stesso costo della rete neurale (slide 15), e questo conferma che la differenza fra i due è tutta nel trasporto.

Risultati con pktgen

4 - I RISULTATI SUL DUAL BOOT - PKTGEN

Con pktgen tutte le pipeline saturano e si ordinano



QUINDI

Sopra la baseline, la rete neurale costa **~40–50 ns** con i pesi compilati (P1, P1.5), **~220 ns** con la forma letta da tabella (P2), **~370 ns** con anche la profondità (P3).

3 giri $\times$ 3 ripetizioni, 64 byte, 28 settembre · variazione fra i giri 0,2–0,9% · DUT a 3 494 MHz, nessun throttling

12

Per ogni programma la slide riporta quanti milioni di pacchetti al secondo regge (la barra), quanti nanosecondi spende per pacchetto (1 diviso la capacità) e quanto costa in più della baseline, cioè il costo della rete neurale. Un core del nodo, tre thread di generatore, pacchetti da 64 byte, 3 giri da 3 ripetizioni.

Esempio di calcolo. P2 regge 1,89 milioni di pacchetti al secondo, quindi spende $1 / 1\,890\,000 \text{ s} \approx 529 \text{ ns}$ per pacchetto. La baseline ne spende 302. La differenza, $529 - 302 \approx 226 \text{ ns}$, è il tempo che P2 dedica alla rete neurale: ricevere, copiare, leggere l'intestazione e inoltrare lo fanno tutte e due allo stesso modo.

Le tre letture della slide:

- **Il costo dipende da dove stanno i pesi e la forma.** Pesi scritti nel codice (P1, P1.5): la rete costa 40–50 ns. Forma letta da tabella (P2): circa 220 ns, perché i cicli girano fino ai soffitti e ogni peso è una lettura da mappa. Anche la profondità da tabella (P3): circa 370 ns, per i salti fra uno strato e l'altro.
- **Sotto capacità, zero perdite:** mandando a tutte 1,37 milioni di pacchetti al secondo (il 91% di quanto regge la più lenta, P3) nessuna respinge un pacchetto: la perdita totale resta entro lo 0,02%, che è rumore di lettura dei contatori.
- **P1 contro P1.5:** 2,94 contro 2,81 milioni al secondo. Congelare il numero del nodo nel codice fa risparmiare circa 15 ns per pacchetto: poco, ma misurabile, perché le ripetizioni variano meno dell'1%.

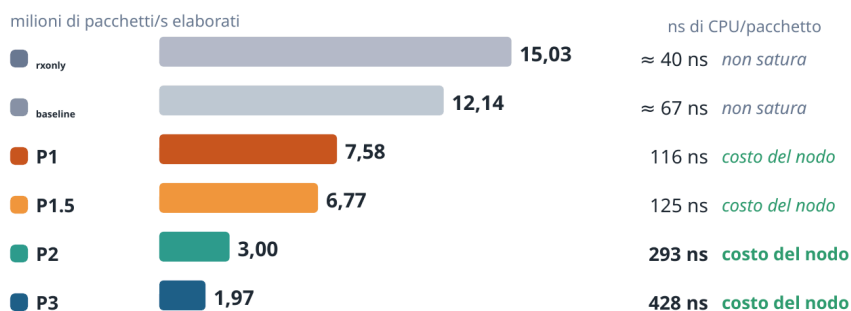
rxonly regge 4,61 milioni al secondo (217 ns): è il tetto di ricezione con pktgen, e tutte le pipeline stanno sotto. La baseline sta al 72% di quel tetto, P3 al 33%.

Verificato il 30 settembre. Con pktgen il core del nodo è occupato al 100%, quindi questi sono tempi di CPU veri. I tre thread aggiungono circa 17 ns di contesa sulla coda, uguali per tutte (baseline 278 ns con un thread contro 296 con tre): i costi **sopra la baseline** non cambiano.

Risultati con xdp_gen

4 - I RISULTATI SUL DUAL BOOT - XDP_GEN

Con xdp_gen, un thread per coda: il costo del nodo



Il tetto era la coda

Con tre thread nella stessa coda il nodo si fermava a ~7 M/s. Con un thread per coda la baseline elabora 12 M/s.

QUINDI

P1, P1.5, P2 e P3 sono **costi del nodo**: 116, 125, 293 e 428 ns di CPU. Baseline e rxonly, con un thread, non saturano: i loro valori sono indicativi.

1 thread, 1 core, uscita separata - 3 giri, 64 byte - costo = tempo di CPU del nodo per pacchetto - a 512 e 1 514 byte P2 e P3 restano entro il 3%

13

Un thread di generatore, un core del nodo, l'uscita su un core suo: **uno scrittore per ogni coda**, come con una scheda di rete vera, dove la coda di ricezione la riempie solo la scheda. La colonna a destra è il **tempo di CPU** che il core del nodo spende per ogni pacchetto, dalla presa dalla coda di ricezione alla decisione e al redirect verso l'uscita.

- **P1, P1.5, P2 e P3 saturano** (una parte dei pacchetti viene respinta): 116, 125, 293 e 428 ns sono costi del nodo.
- **Baseline e rxonly non saturano**: un thread offre 12-15 milioni al secondo e il nodo li prende tutti. Il loro tempo di CPU (circa 67 e 40 ns) è misurato con pochi pacchetti per giro di ricezione, ed è indicativo. La misura a due core della slide 14, dove la baseline satura, dà 68 ns per core e lo conferma.

Perché il tempo di CPU. Un thread solo spedisce a raffiche: la raffica riempie la coda e il resto viene respinto, poi fra una raffica e l'altra il nodo svuota la coda e aspetta. Il core del nodo lavora così l'84-88% del tempo, anche mentre respinge il 70-80% dei pacchetti. 1 diviso i pacchetti elaborati conterebbe anche l'attesa: P2 risulterebbe 334 ns invece di 293. Il tempo di CPU moltiplica per l'occupazione del core e toglie l'attesa.

Come si è capito che il tetto era la coda. Con tre thread tutti i programmi leggeri si fermavano intorno a 7 milioni al secondo. Togliendo thread il nodo elabora **di più**: la baseline 7,2 milioni con tre thread, 7,3 con due, 12,4 con uno. I thread che scrivono nella stessa coda si passano un lucchetto e le stesse righe di cache, e il core del nodo che la svuota ne subisce il rallentamento. P2 e P3, il cui limite è il programma, quasi non ne risentono.

Verificato il 30 settembre: scartare costa più che inoltrare, circa 50 ns: nella misura per classe la classe DROP costa 173 ns su P1 contro 117, 480 su P3 contro 427. Quando il pacchetto viene scartato la sua memoria si libera sul core del nodo; quando viene inoltrato, su quello d'uscita. A 512 e 1 514 byte P2 e P3 costano come a 64 entro il 3%: il nodo tocca solo le intestazioni.

Con xdp_gen due core reggono il doppio

4 - I RISULTATI SUL DUAL BOOT - XDP_GEN

Con xdp_gen due core reggono il doppio

Milioni di pacchetti al secondo elaborati, con xdp_gen

■ 2 core, misurati ■ 2 × un core



Una coda d'uscita per ogni core

Prima i due core del nodo scrivevano nella stessa coda d'uscita, svuotata da un solo core: la baseline elaborava 13,1 M/s e ne arrivavano 4,3. Ora ogni core ha la sua coda e il suo core d'uscita: arrivano tutti.

Il costo per core non cambia

Tempo di CPU per pacchetto di ogni core: 68, 117, 127, 298 e 434 ns, contro 67, 116, 125, 293 e 428 con un core solo (slide 13).

QUINDI

Con uno scrittore per coda anche in uscita, due core elaborano il **95-99% del doppio** di un core: il costo misurato su un core vale per ogni core.

xdp_gen, 1 ottobre, 64 byte, 3 giri · nodo su CPU 6 e 8, un thread per coda, uscita su CPU 3 e 5 · 2 × un core = 2 × tempo di CPU della slide 13

14

La slide dà al nodo **due core** (CPU 6 e 8), ciascuno con la sua coda d'ingresso alimentata da un thread di xdp_gen. Le barre colorate sono i pacchetti al secondo elaborati con due core; le barre grigie sono il riferimento: **il doppio di quello che regge un core**, calcolato come 2 diviso il tempo di CPU della slide 13 (per P2: $2 / 293 \text{ ns} = 6,83$ milioni al secondo).

Come si legge. P2 con due core elabora 6,68 milioni al secondo, il riferimento è 6,83: il 98%. Le altre: baseline 28,33 contro 29,85 (95%, ma il riferimento della baseline viene dal valore indicativo di 67 ns), P1 17,05 contro 17,24 (99%), P1.5 15,81 contro 16,00 (99%), P3 4,61 contro 4,67 (99%).

Il riquadro in basso a destra lo dice nell'unità del costo: il tempo di CPU per pacchetto di ogni core è 68, 117, 127, 298 e 434 ns, contro 67, 116, 125, 293 e 428 con un core solo. Il secondo core non rallenta il primo: i contatori delle pipeline sono per-core (slide 23), quindi i due core non si contendono niente di scritto a ogni pacchetto. Al nodo successivo arrivano tutti i pacchetti elaborati (persi dopo XDP: 0,03% sulla baseline, zero sulle altre).

Perché serve una coda d'uscita per ogni core. Il riquadro in alto a destra racconta il problema che il banco aveva fino al 30 settembre. Il redirect verso il cavo d'uscita sceglie la coda dal numero del core che inoltra (numero modulo numero di code). Il banco dava a ogni porta d'uscita **una coda sola**: i due core del nodo scrivevano nello stesso anello, e un solo core del nodo successivo lo svuotava. Due danni insieme:

1. **contesa**: lucchetto e righe di cache dell'anello passano da un core all'altro a ogni pacchetto, e il costo lo paga il core del nodo nel redirect;
2. **un solo core d'uscita non basta**: ricevere, contare e restituire la pagina costa al nodo successivo circa 70 ns a pacchetto, cioè un tetto di circa 10 milioni al secondo. Oltre, l'anello trabocca, il redirect fallisce e il pacchetto si perde dopo il nodo, che paga anche lo scarto.

Il problema e la soluzione, un passo alla volta (baseline, milioni al secondo elaborati e arrivati, tempo di CPU per core). Una coda d'uscita e un core d'uscita: 13,15 elaborati, 4,2 arrivati (49% perso dopo XDP), 152 ns. Tre code d'uscita (il core 6 scrive nella 0, l'8 nella 2) e un core d'uscita: 19,15 elaborati, ne arrivano circa 10, 103 ns. Tre code e due core d'uscita (CPU 3 e 5, uno per coda): 28,33 elaborati, 28,33 arrivati, 68 ns. P2 e P3, più lente dell'uscita, valevano già 6,6 e 4,6 in tutti e tre i casi.

Perché tre code e non due? Con due code i core 6 e 8 cadrebbero tutti e due nella coda 0 (6 e 8 sono entrambi pari): il banco sceglie il numero minimo di code per cui i core del nodo cadono in code diverse. Con un solo core del nodo resta una coda, quindi le misure a un core (slide 12 e 13) non cambiano.

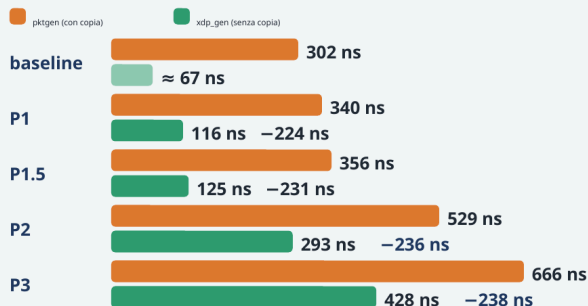
Verificato il 1 ottobre. Il banco controlla a fine run quali thread d'uscita hanno lavorato e su quale core: hanno lavorato solo la coda 0 su CPU 3 e la coda 2 su CPU 5, come previsto. Il run con una coda sola riproduce entro l'1% quello del 30 settembre. La baseline ora satura appena (il generatore offre 29 milioni al secondo e ne viene respinto il 2%): la sua cifra è vicina al tetto del generatore.

La differenza fra i due generatori

4 · I RISULTATI SUL DUAL BOOT · LE DIFFERENZE

Con xdp_gen lo stesso programma costa ~230 ns in meno

Nanosecondi per pacchetto, stesso programma



Con pktgen si citano tutte e cinque

Costo sopra la baseline: +38, +53, +226 e +364 ns per P1, P1.5, P2 e P3.

Con xdp_gen si citano P1, P1.5, P2 e P3

116, 125, 293 e 428 ns di CPU, un thread per coda. Baseline ≈ 67 ns: indicativa.

Da dove vengono i ~230 ns

Dalla copia della skb e dal percorso dello stack: il costo del generatore, pagato sul core del nodo.

QUINDI

I due generatori **non si confrontano** fra loro: ognuno confronta le pipeline al suo interno. La differenza è il costo del percorso skb.

pktgen del 28 settembre, xdp_gen del 30 (tempo di CPU) · P1: 340 - 116 = 224 · P1.5: 356 - 125 = 231 · P2: 529 - 293 = 236 · P3: 666 - 428 = 238 ns

15

Lo stesso programma misurato con i due generatori, su un core. La barra arancione (pktgen) è sempre più lunga di quella verde (xdp_gen): la differenza è il costo della copia della skb e del percorso nel kernel, pagato dal core del nodo prima che il programma parta.

Esempio. P2: 529 ns con pktgen, 293 ns con xdp_gen, differenza 236 ns. P3: 666 contro 428, differenza 238 ns. P1: 340 contro 116, differenza 224. P1.5: 356 contro 125, differenza 231. La differenza è praticamente uguale per tutte le pipeline: è un costo fisso del trasporto, che non dipende dalla rete neurale.

I due riquadri a destra dicono che cosa si cita con ciascun generatore:

- **con pktgen** tutte e cinque, con il costo sopra la baseline: +38, +53, +226 e +364 ns per P1, P1.5, P2 e P3;
- **con xdp_gen** P1-P3 in tempo di CPU: 116, 125, 293 e 428 ns. La baseline (circa 67 ns) è indicativa su un core; su due core, dove satura, vale 68 ns per core (slide 14).

Conclusione: i due generatori non si confrontano fra loro. Ciascuno serve a confrontare le pipeline fra di loro, con lo stesso trasporto per tutte. Le cifre di pktgen sono del 28 settembre, quelle di xdp_gen del 30 (tempo di CPU, un thread per coda). La cifra dei circa 230 ns è un calcolo fatto da questi numeri.

Come si risponde alla domanda: due programmi in catena

5 · LA DOMANDA DEL RELATORE · COME SI MISURA

Due programmi sullo stesso aggancio, tre conteggi

Un'interfaccia porta un solo programma XDP: i due si mettono **in catena**, con un salto.



inviati – ricevuti

= persi **prima** del programma
respinti dalla coda d'ingresso

ricevuti – inoltrati

= persi **dopo** il programma
o decisioni del modello (scarta)

rxonly = il solo programma 1

Dice quanto regge la sola ricezione allo
stesso ritmo. Il programma 1 costa fino a
+5 ns: dentro il rumore.

Il salto è la tail call che le pipeline già usano · costo del programma 1 misurato con le statistiche del kernel

16

Per sapere dove si perdono i pacchetti servono tre contatori, messi in tre punti diversi del percorso:

1. **inviati**: li conta il generatore;
2. **ricevuti**: li conta un piccolo programma messo davanti alla pipeline, che fa solo `ing_count += 1` e poi passa il pacchetto alla pipeline;
3. **inoltrati**: li conta il nodo successivo.

Un'interfaccia può avere un solo programma XDP agganciato. Per averne due, il primo (il contatore) salta al secondo (la pipeline) con una **tail call**, lo stesso meccanismo che le pipeline usano già al loro interno (P3 lo usa fra uno strato e l'altro). Il salto non torna indietro: la pipeline prosegue come se fosse stata agganciata direttamente.

Con i tre conteggi la perdita si divide in due parti:

- **inviati – ricevuti** = persi prima del programma, cioè respinti dalla coda d'ingresso piena;
- **ricevuti – inoltrati** = persi dopo il programma, oppure pacchetti che il modello ha deciso di scartare (si separano con i contatori della pipeline, slide 17).

Esempio. In una finestra di misura: inviati 1 000 000, ricevuti 800 000, inoltrati 800 000. Allora 200 000 pacchetti si sono persi prima del programma (respinti dalla coda) e zero dopo.

Il contatore costa pochissimo, e lo si misura invece di supportarlo: il kernel tiene il tempo di ogni programma agganciato, e con e senza il contatore davanti la differenza va da -2 a +5 ns, dentro la variazione fra le ripetizioni. rxonly è il contatore da solo, senza pipeline dietro: dice quanto regge la sola ricezione allo stesso ritmo.

Come si leggono i tre conteggi

5 · LA DOMANDA DEL RELATORE · COME SI LEGGE

Dove mancano i pacchetti indica il collo di bottiglia

Un programma XDP non ha una coda sua: se è lento, si riempie la coda che sta **davanti** a lui.

SI OSSERVA

inviati > ricevuti

mancano pacchetti prima di XDP



VUOL DIRE

La coda d'ingresso è piena: il collo di bottiglia è **il programma** (o la sola ricezione: lo separa rxonly).

SI OSSERVA

ricevuti > inoltrati

al nodo successivo ne arrivano meno: si guarda che cosa aveva deciso il programma



SE AVEVA DECISO DI INOLTARE

Il pacchetto non è arrivato: il collo di bottiglia è **la trasmissione**.

SE AVEVA DECISO DI SCARTARE (O NON USATA)

Non è una perdita: è **una decisione** della rete neurale.

QUINDI

Tre conteggi più le decisioni del programma separano **tre casi** che altrimenti sembrerebbero uguali.

Inviati: generatore · ricevuti: programma 1 · inoltrati: nodo successivo · decisioni: contatori della pipeline (HIT, MISS, DROP)

17

Un programma XDP non ha una coda sua: elabora un pacchetto alla volta, dall'inizio alla fine, sul core che lo ha ricevuto. Se è lento, non accumula lavoro dentro di sé: si riempie la coda che sta davanti a lui. Per questo il punto in cui mancano i pacchetti dice chi è il collo di bottiglia. La slide distingue tre casi:

- **inviati > ricevuti**: mancano pacchetti prima di XDP, la coda d'ingresso era piena. Il collo di bottiglia è il programma, oppure la sola ricezione: lo si capisce confrontando con rxonly (slide 21).
- **ricevuti > inoltrati, e il programma aveva deciso di inoltrare**: il pacchetto è stato elaborato ma non è arrivato. Il collo di bottiglia è la trasmissione in uscita.
- **ricevuti > inoltrati, e il programma aveva deciso di scartare** (o la classe non è usata): non è una perdita, è la rete neurale che ha scelto così.

Le decisioni si leggono dai contatori che ogni pipeline tiene comunque: HIT (inoltrati secondo il modello), MISS (nessuna regola trovata) e DROP (scartati per decisione).

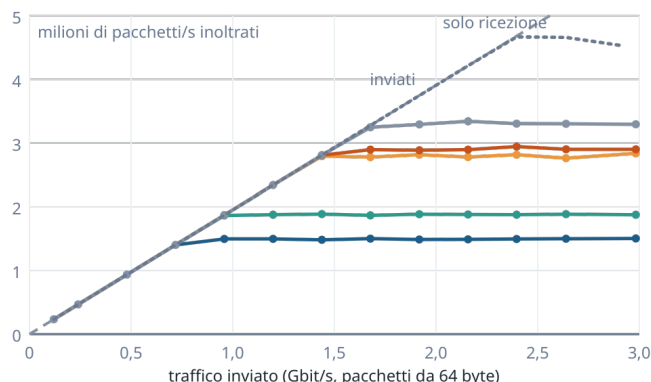
Esempio. Ricevuti 100, inoltrati 90. Se il programma ha deciso "scarta" per 10 pacchetti, va tutto bene: zero perdite. Se invece aveva deciso "inoltra" per tutti e 100, 10 si sono persi in uscita.

Tre conteggi più le decisioni del programma separano quindi tre casi che, guardando solo inviati e arrivati, sembrerebbero uguali.

Le curve: pacchetti inoltrati al crescere del traffico

5 · LA DOMANDA DEL RELATORE · PACCHETTI INOLTRATI

Ogni curva sale con il traffico e poi si ferma



Dove si ferma ogni pipeline

pipeline	pulita fino a (Gbit/s)	perde da (Gbit/s)	inoltro max (M pacchetti/s)
baseline	1,44	1,68	3,34
P1	1,20	1,44	2,94
P1.5	1,20	1,44	2,84
P2	0,72	0,96	1,89
P3	0,72	0,96	1,50

Sono gradini della scala (~0,24 Gbit/s), non soglie esatte. A 64 byte, 1 Gbit/s $\approx$ 1,95 milioni di pacchetti al secondo.

QUINDI

Ogni curva si ferma alla **capacità della sua pipeline**: 3,3 · 2,9 · 2,8 · 1,9 · 1,5 milioni di pacchetti al secondo, la stessa cifra del test di saturazione.

13 valori di traffico, 5 giri ciascuno, 64 byte, 390 finestre · DUT a 3 494 MHz, nessun throttling

18

Come si legge il grafico. In orizzontale c'è il traffico inviato (in Gbit/s, con pacchetti da 64 byte), in verticale i pacchetti che il nodo riesce a inoltrare (milioni al secondo). La linea tratteggiata grigia è "tutto quello che si invia": finché una curva sta sulla diagonale, il nodo inoltra tutto. Quando la curva diventa piatta, il nodo è arrivato alla sua capacità e il resto si perde.

Da Gbit/s a pacchetti. Un pacchetto da 64 byte è 512 bit. Quindi $1 \text{ Gbit/s} = 1\,000\,000\,000 / 512 \approx 1,95$ milioni di pacchetti al secondo. P2 si ferma a 1,89 milioni di pacchetti al secondo, cioè $1,89 \times 0,512 \approx 0,97$ Gbit/s.

Le cinque pipeline si fermano alla capacità trovata nel test di saturazione (slide 12): 3,3, 2,9, 2,8, 1,9 e 1,5 milioni al secondo. Due test diversi, uno a traffico crescente e uno a traffico massimo, danno lo stesso numero: è una conferma che la capacità misurata è una proprietà del nodo e non del metodo. La curva tratteggiata in alto (rxonly) si ferma molto più in là, a circa 2,4 Gbit/s.

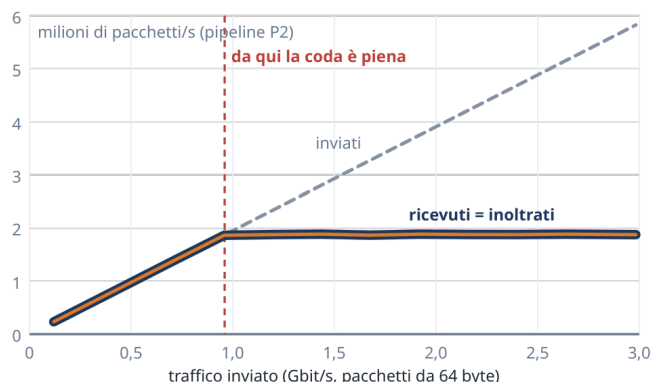
Le colonne della tabella. Il traffico si prova solo a 13 valori, distanti circa 0,24 Gbit/s. "Pulita fino a" è l'ultimo valore provato senza perdite, "perde da" il primo con perdite: la soglia vera sta fra i due. Esempio: P2 è pulita a 0,72 e perde a 0,96, e la sua soglia vera è 0,97 (proprio sul gradino). P3 ha gli stessi gradini di P2: la differenza fra le due si legge nell'ultima colonna (1,89 contro 1,50).

Ogni punto è la mediana di 5 giri: $13 \text{ valori} \times 5 \text{ giri} \times 6 \text{ programmi} = 390$ finestre di misura, tutte con il DUT a 3 494 MHz e senza throttling.

Ricevuti e inoltrati coincidono

5 · LA DOMANDA DEL RELATORE · RICEVUTI E INOLTRATI

Il programma rilancia tutto ciò che riceve



1

Sotto il limite, le curve coincidono

Tutto ciò che si invia viene ricevuto e inoltrato.

2

Sopra il limite, salgono solo gli inviati

Ricevuti e inoltrati si fermano insieme, a 1,9 milioni al secondo.

3

Ciò che manca non è mai entrato

Il programma non perde niente di quello che prende.

QUINDI

Ricevuti e inoltrati restano sovrapposti in tutte e cinque le pipeline: dopo XDP si perde al massimo lo 0,02% degli inviati.

Esempio: P2, mediana di 5 giri · linea scura = ricevuti da XDP, arancione = inoltrati; coincidono

19

Il grafico mostra una sola pipeline, P2, con tre curve: inviati (tratteggiata), ricevuti dal programma (linea scura) e inoltrati (arancione). Ricevuti e inoltrati sono una sopra l'altra per tutto il grafico: tutto quello che il programma riceve, lo rilancia.

Si leggono tre fasi:

1. **sotto il limite** le tre curve coincidono: tutto ciò che si invia viene ricevuto e inoltrato;
2. **sopra il limite** sale solo la curva degli inviati; ricevuti e inoltrati si fermano insieme a 1,9 milioni al secondo;
3. **ciò che manca non è mai entrato**: la distanza fra inviati e ricevuti è tutta perdita all'ingresso.

Esempio con i numeri veri. A 1,44 Gbit/s si inviano 2,81 milioni di pacchetti al secondo. P2 ne riceve 1,89 e ne inoltra 1,89. I 0,92 milioni mancanti si sono persi tutti prima del programma, alla coda d'ingresso.

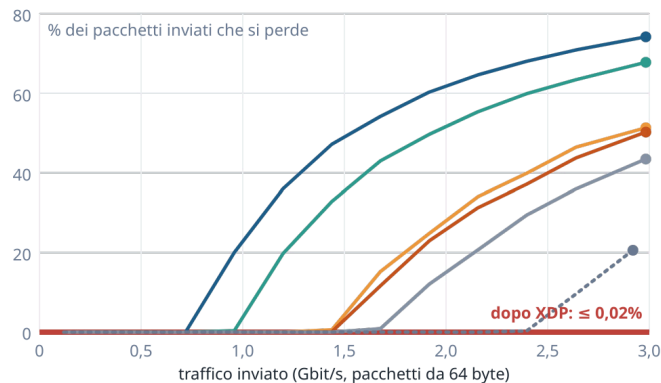
La linea rossa verticale segna il punto da cui la coda è piena: a sinistra le tre curve coincidono, a destra sale solo quella degli inviati. Lo stesso vale per tutte e cinque le pipeline, non solo per P2.

Sullo "0,02%". Non è una perdita vera: è rumore di misura. Il banco legge i contatori uno dopo l'altro, non tutti nello stesso istante, e in quel breve intervallo passano alcune centinaia di pacchetti. Su 325 misure il valore è negativo nel 47% dei casi (una perdita vera non può essere negativa) e in media è zero. Più corretto dire: dopo il programma la perdita è zero, entro il rumore di misura.

La perdita è tutta prima del programma

5 · LA DOMANDA DEL RELATORE · LA PERDITA

La perdita è tutta prima del programma



Perdita a circa 3 Gbit/s

prima del programma, % degli inviati

P3	74%
P2	68%
P1.5	51%
P1	50%
baseline	43%
rxonly (solo ricezione)	21%

Dopo il programma: ≤ 0,02%

in tutte e cinque le pipeline, a ogni traffico

QUINDI

Se si perde all'ingresso il collo di bottiglia è il programma: qui la perdita è **tutta all'ingresso**, quindi il collo di bottiglia non è la trasmissione.

Perdita = (inviati - inoltrati) / inviati · mediana di 5 giri · a circa 3 Gbit/s si inviano 5,8 M pacchetti/s (5,7 per rxonly)

20

Qui si guarda la percentuale di pacchetti persi, cioè $(\text{inviati} - \text{inoltrati}) / \text{inviati}$. Ogni curva resta a zero finché la pipeline regge, poi sale: più traffico si manda oltre la capacità, più grande è la parte che non entra. La curva sale tanto prima quanto più la pipeline è costosa.

Esempio di calcolo. A circa 3 Gbit/s si mandano 5,8 milioni di pacchetti al secondo. P3 ne elabora 1,5 milioni. La parte persa è $1 - 1,5 / 5,8 \approx 74\%$. La baseline ne elabora 3,3 milioni: $1 - 3,3 / 5,8 \approx 43\%$.

Più la pipeline è costosa, più perde a parità di traffico: P3 (74%) > P2 (68%) > P1.5 (51%) > P1 (50%) > baseline (43%) > rxonly (21%). L'ordine è lo stesso della capacità della slide 12.

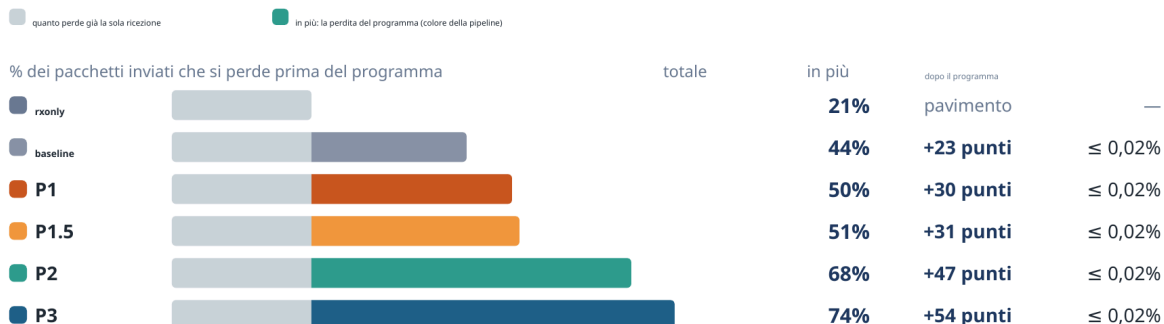
La linea rossa in basso è la perdita dopo il programma: resta a zero per tutte le pipeline e a ogni traffico (lo "0,02%" è rumore, vedi slide 19). Quindi il collo di bottiglia è il programma, non la trasmissione.

È il programma o la sola ricezione?

5 · LA DOMANDA DEL RELATORE · IL PROGRAMMA O LA RICEZIONE?

Oltre la sola ricezione, la perdita è del programma

A circa 3 Gbit/s anche il solo contatore (rxonly) perde il 21%: è il **pavimento**.



QUINDI

La perdita in più cresce con il **costo del programma**. Fino a 2,4 Gbit/s la sola ricezione non perde, e dopo il programma non si perde niente.

Traffico più alto della scala: 5,8 M pacchetti/s (5,7 per rxonly) · punti = differenza fra le due percentuali

21

Anche rxonly, che non fa niente oltre a ricevere, a circa 3 Gbit/s perde il 21%: 5,7 milioni di pacchetti al secondo sono troppi anche solo da ricevere (con pktgen la sola ricezione regge circa 4,6 milioni). Quel 21% è il **pavimento**: la parte di perdita che non dipende dalla pipeline.

Ogni barra ha quindi due pezzi: quello grigio è quanto perde già la sola ricezione, quello colorato è la perdita **in più** rispetto al pavimento, dovuta al lavoro del programma. La colonna "dopo il programma" resta a zero per tutte.

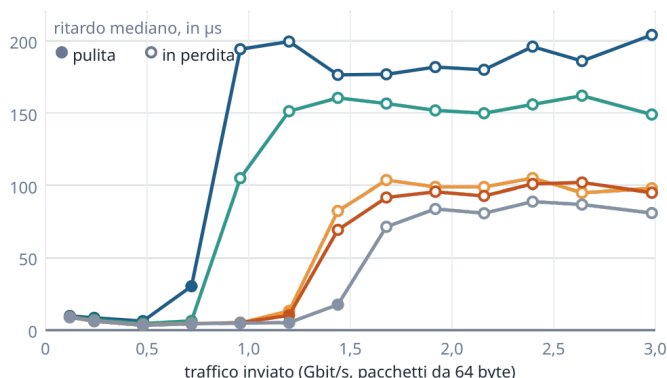
Esempio. La baseline perde il 44%, rxonly il 21%. La differenza, $44 - 21 = 23$ punti, è dovuta al programma. Per P3: $74 - 21 = 54$ punti. La perdita in più cresce con il costo del programma: +23, +30, +31, +47, +54 punti dalla baseline a P3.

C'è anche un secondo argomento: fino a 2,4 Gbit/s rxonly non perde niente, mentre le pipeline cominciano a perdere molto prima (P2 e P3 già sotto 1 Gbit/s). Quindi nella zona in cui le pipeline cominciano a perdere, la ricezione da sola ce la fa ancora: il limite è davvero il programma.

Il ritardo

5 · LA DOMANDA DEL RELATORE · IL RITARDO

Il ritardo salta dove comincia la perdita



La coda piena spiega il ritardo

attesa prevista = 256 posti + capacità della pipeline

pipeline	prevista	misurata (p50)
baseline	77 μs	84 μs
P1	87 μs	95 μs
P1.5	90 μs	99 μs
P2	135 μs	154 μs
P3	171 μs	186 μs

Il resto (circa il 10%) è il percorso fino al nodo successivo.

QUINDI

Oltre la sua capacità il nodo non riesce a svuotare la coda, che resta piena: il ritardo si ferma e cresce la perdita.

Ritardo dal timbro del generatore all'arrivo al nodo successivo · mediana (p50) oltre la saturazione · capacità come nella slide 18

22

Il ritardo è il tempo dal momento in cui il generatore crea il pacchetto (lo scrive nel pacchetto stesso, come un timbro) a quando arriva al nodo successivo. Il grafico mostra la mediana, cioè il valore "tipico".

- Finché il nodo regge (pallini pieni) il ritardo è di pochi microsecondi: la coda è quasi vuota e ogni pacchetto viene elaborato subito.
- Quando comincia la perdita (pallini vuoti) il ritardo salta di colpo a 80–200 μs : la coda è piena, e ogni pacchetto deve aspettare che passino i 255 davanti a lui.
- Poi resta piatto: la coda non può diventare più lunga di 256 posti. Il traffico in più non aumenta l'attesa, viene semplicemente respinto.

Esempio di calcolo. P2 elabora 1,89 milioni di pacchetti al secondo, cioè uno ogni 529 ns. Con la coda piena un pacchetto aspetta $256 \times 529 \text{ ns} \approx 135 \mu\text{s}$. La misura dà 154 μs : il resto (circa il 10%) è il viaggio fino al nodo successivo.

La tabella a destra fa lo stesso conto per tutte le pipeline (attesa prevista = 256 posti diviso la capacità): baseline 77 contro 84 μs misurati, P1 87 contro 95, P1.5 90 contro 99, P2 135 contro 154, P3 171 contro 186. Previsto e misurato sono sempre vicini, quindi il salto del ritardo è proprio la coda piena.

Il ritardo dà quindi la stessa risposta della perdita: oltre la sua capacità il nodo non riesce a svuotare la coda, che resta piena; il ritardo si ferma al livello della coda piena e cresce solo la perdita.

Con due core la capacità raddoppia

5 · LA DOMANDA DEL RELATORE · PIÙ CORE

Con due core la capacità raddoppia

Capacità massima, in milioni di pacchetti al secondo



Un contatore per ogni core

Le statistiche sono per-core: ogni core incrementa la sua copia, senza istruzioni atomiche e senza contendersi una riga di cache. Chi legge somma.

Quattro thread, due per coda

Con tre thread una coda ne aveva uno solo (~2,8 M/s): rxonly non saturava (×1,64).

Una coda e un core, per scelta

Il banco misura il costo per core. Una scheda vera distribuisce i pacchetti su più code e più core (RSS).

QUINDI

Il costo per pacchetto resta quello di un core: un nodo con N code regge **circa N volte** la capacità misurata su un core, con pktgen e con xdp_gen (slide 14).

pktgen a 4 thread, 2 per coda · una coda d'uscita per core (CPU 6 e 8) · 64 byte, 3 giri · 1 core: 28/9, 2 core: 1/10

23

Finora il nodo usava un solo core e una sola coda, apposta, per misurare il costo per core. Qui si danno al nodo due core (CPU 6 e 8), ciascuno con la sua coda d'ingresso e la sua coda d'uscita, con pktgen a quattro thread, due per coda. Le pipeline elaborano il doppio: da 1,93 a 2,04 volte.

Esempio. P3 passa da 1,49 a 2,88 milioni di pacchetti al secondo (×1,93), la baseline da 3,40 a 6,72 (×1,98), P1 da 3,01 a 6,02 (×2,00). Sopra la baseline ogni core paga quello che pagava da solo: +34, +58, +231 e +397 ns per P1, P1.5, P2 e P3.

Il riquadro "Un contatore per ogni core" spiega perché funziona: i contatori della pipeline (pacchetti inoltrati, persi, scartati, e uno per classe) sono **per-core**. Ogni core incrementa la sua copia, senza istruzioni atomiche e senza contendersi una riga di cache; chi legge somma le copie. Se fossero un contatore unico, i due core dovrebbero passarselo a ogni pacchetto e si rallenterebbero a vicenda.

Il riquadro "Quattro thread, due per coda" spiega perché i thread sono quattro. Il thread pktgen numero i scrive nella coda i modulo 2. Con tre thread la coda 0 riceve da due thread (circa 5,5 milioni al secondo offerti) e la coda 1 da uno solo (circa 2,8). Per le pipeline 2,8 milioni bastano a tenere pieno il core; per rxonly no, perché un core riceve circa 4,7 milioni al secondo. Il core della coda 1 restava fermo il 40% del tempo, e rxonly si fermava a $4,75 + 2,77 \approx 7,5$ milioni (×1,64). Con quattro thread le due code ricevono lo stesso traffico e rxonly arriva a 9,41 milioni (×2,04).

Le due correzioni del 1 ottobre, con pktgen. Prima i due core scrivevano in una coda d'uscita sola (come nella slide 14): costava 12–20 ns per pacchetto per core, uguali per tutte, e i rapporti erano 1,91–1,98. Con una coda d'uscita per core la baseline passa da 6,52 a 6,92 milioni al secondo (+6%), P2 da 3,74 a 3,87 (+3%), rxonly, che non inoltra, non cambia: i costi sopra la baseline restano uguali. Il passaggio a quattro thread toglie il limite di rxonly e costa alle pipeline il 2–4% (ora tutte e due le code hanno due scrittori).

Il terzo riquadro ricorda che **una coda e un core sono una scelta**: il banco misura il costo per core. Una scheda di rete vera distribuisce i pacchetti su più code e più core (RSS), con un solo scrittore per coda per costruzione (la scheda stessa), e la capacità del nodo cresce con il numero di core.

Conclusione: il costo per pacchetto resta quello misurato su un core, con pktgen (qui) e con xdp_gen (slide 14). Un nodo con N code regge circa N volte la capacità di un core, finché generatore e scheda reggono.

In sintesi

IN CHIUSURA · SINTESI

In sintesi

1

Il dual boot rende le misure ripetibili

La variazione del costo scende da 15-160% a 0,3-0,6%. Il core del nodo non dorme, non cambia frequenza e nessuno lo interrompe.

2

Due generatori, due usi

pktgen confronta tutte le pipeline, da 3,31 a 1,50 M/s. xdp_gen, un thread per coda, misura P1-P3 senza la copia: 116-428 ns.

3

Si perde prima del programma

Il nodo satura alla sua capacità, il ritardo salta di un ordine di grandezza e la perdita è tutta all'ingresso: il collo di bottiglia è il programma, e due core reggono il doppio.

Ettore Cantile · settembre 2026

25

- 1. Il dual boot rende le misure ripetibili.** Nella macchina virtuale lo stesso test dava risultati molto diversi fra una volta e l'altra (P1 variava del 160%); ora le ripetizioni coincidono entro pochi punti, e si vedono anche differenze piccole come P1 contro P1.5 (15 ns) o scartare contro inoltrare (50 ns).
- 2. Due generatori, due usi.** pktgen serve a confrontare tutte le pipeline (da 3,31 a 1,50 milioni di pacchetti al secondo su un core). xdp_gen, con un thread per coda, dà il costo senza la copia per P1-P3 (116, 125, 293 e 428 ns di CPU). Sopra la baseline la rete neurale costa uguale con i due.
- 3. Si perde prima del programma.** Quando il traffico supera la capacità, la coda si riempie, il ritardo salta di un ordine di grandezza e i pacchetti vengono respinti all'ingresso; dopo il programma non se ne perde nessuno. Il collo di bottiglia è il programma, e due core reggono il doppio di uno, con entrambi i generatori.

Cose da dire con cautela. Slide 8: la variazione "0,5%" non si confronta direttamente con quella della macchina virtuale (vedi la nota della slide 8). Slide 19, 20 e 21: " $\leq 0,02\%$ " è rumore di misura, meglio dire "zero". Slide 14: la baseline a due core satura appena, la sua cifra è vicina al tetto del generatore. Slide 23: le cifre a un core sono del 28 settembre, quelle a due core del 1 ottobre (sessioni diverse, variazione fra sessioni circa 1-3%).

Risolve il 30 settembre e il 1 ottobre. Il "tetto del generatore" con xdp_gen non era del generatore ma della coda d'ingresso condivisa: le cifre di xdp_gen sono ora con un thread per coda e in tempo di CPU (slide 10, 11, 13, 15). Con due core i core del nodo scrivevano in una coda d'uscita sola, svuotata da un solo core: ora c'è una coda d'uscita per core e, con xdp_gen, un core d'uscita per coda (slide 14, 23). Con pktgen a due core i thread sono quattro, due per coda, e anche rxonly raddoppia (slide 23). Slide 18: la riga P3 è corretta (0,72 / 0,96).